Configure Allowed HTTP Methods Globally in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorials

How to Configure the Allowed HTTP Methods Globally in ASP.NET Core Web API

In this article, I will discuss **How to Configure the Allowed HTTP Methods Globally in ASP.NET Core Web API** application with Example. Please read our previous article discussing **How to Add Custom HTTP Status Code in ASP.NET Core Web API** with Examples.

How to Configure Allowed HTTP Methods Globally in ASP.NET Core Web API?

Configuring the allowed HTTP methods globally in an ASP.NET Core Web API project is essential for defining which HTTP request methods (such as GET, POST, PUT, DELETE) are accepted by the application. This configuration helps enhance the security and manageability of our API by ensuring that only intended methods are handled by your application. There are several approaches to achieve this configuration.

Using Custom Middleware to Configure Allowed HTTP Methods Globally

You can create custom middleware that intercepts the HTTP method of each request and evaluates whether it is one of the allowed methods. If the method is not in the allowed list, the middleware can return a 405 Method Not Allowed status code. This approach allows you to centralize the logic for handling HTTP method validation and ensure that only the specified HTTP methods reach the application. Requests that are disallowed methods are rejected with a 405 Method Not Allowed status code. The following middleware ensures that only GET, POST, PUT, and DELETE methods are allowed.

```
namespace DotNetTutorialsMinimalWebAPI.Middlewares
{
    public class AllowedHttpMethodsMiddleware
    {
        private RequestDelegate _next;
        private HashSet<string> _allowedMethods;

        public AllowedHttpMethodsMiddleware(RequestDelegate next, string[] allowedMethods)
        {
            _next = next;
            _allowedMethods = new HashSet<string>(allowedMethods, StringComparer.OrdinalIgnoreCase);
            // Initialize the allowed methods with the provided array
            // Using HashSet for efficient lookup and case-insensitive comparison
        }

        public async Task Invoke(HttpContext context)
        {
            // Check if the request method is in the list of allowed methods
            if (!_allowedMethods.Contains(context.Request.Method))
            {
                // If the method is not allowed, return 405 Method Not Allowed
                context.Response.StatusCode = 405;
                return;
                // Exit the pipeline early to prevent further processing
            }
            // If the method is allowed, continue to the next middleware
            await _next(context);
            // Call the next middleware in the pipeline
            // This allows the request to be processed by other middleware
            // If the method is not allowed, this will not be reached
        }
    }
    public static class AllowedHttpMethodsMiddlewareExtensions
    {
        public static IApplicationBuilder UseAllowedHttpMethods(this IApplicationBuilder app, string[] allowedMethods)
        {
            return app.UseMiddleware<AllowedHttpMethodsMiddleware>(allowedMethods);
            // Extension method to add the middleware to the pipeline
            // Takes the allowed methods as a parameter
        }
    }
}
```

The Middleware checks if the HTTP method of incoming requests is allowed. If it is not allowed, it returns a 405 Method Not Allowed status code. It uses a HashSet for efficient lookup and ignores case sensitivity in HTTP methods. The extension method simplifies adding the middleware to the pipeline.

Register the Middleware

After creating the middleware, you need to register it in the Program class. It is important to add it at the right location in the middleware pipeline to ensure it functions before processing any other requests. Here, we need to specify which HTTP methods are allowed.

```
var app = builder.Build();
app.UseAllowedHttpMethods(new[] { "GET", "POST", "PUT", "DELETE" });
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Creating Controller

Create the following controller. We have created the following controller using Any attribute HTTP methods.

```
namespace DotNetTutorialsMinimalWebAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class HttpMethodsTestController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetTest()
        {
            return Ok("GetTest Data processed");
        }
        [HttpPost]
        public IActionResult PostTest()
        {
            return Ok("PostTest Data processed");
        }
        [HttpPut]
        public IActionResult PutTest()
        {
            return Ok("PutTest Data processed");
        }
        [HttpDelete]
        public IActionResult DeleteTest()
        {
            return Ok("DeleteTest Data processed");
        }
        [HttpOptions]
        public IActionResult OptionsTest()
        {
            return Ok("OptionsTest Data processed");
        }
        [HttpHead]
        public IActionResult HeadTest()
        {
            return Ok("HeadTest Data processed");
        }
        [HttpPatch]
        public IActionResult PatchTest()
        {
            return Ok("PatchTest Data processed");
        }
        [HttpAny]
        public IActionResult AnyTest()
        {
            return Ok("AnyTest Data processed");
        }
    }
}
```

Now, run the application and access the above endpoints. You will see that GET, POST, and DELETE requests are accepted, as shown in the below image.

On the other hand, PUT and PATCH requests are going to return 405 Method Not Allowed responses, as shown in the below image.

Global Action Filter to Handle 405 Status Code

You can create a global action filter that logs HTTP method mismatch or incoming requests and denotes whether to allow or block them. This filter can also be added to the ASP.NET pipeline in the **Program.cs** class file. Its create a class file named **AllowHttpMethodsFilter.cs** and add the following code. The following code is self-explanatory as always read the comments.

```
namespace DotNetTutorialsMinimalWebAPI.Filters
{
    public class AllowHttpMethodsFilter : IAsyncActionFilter, IOrderedFilter
    {
        private ILogger<AllowHttpMethodsFilter> _logger;
        public int Order => 1000;
        // Set the order of the filter to 1000
        // Lower numbers execute first, higher numbers execute later
        public AllowHttpMethodsFilter(ILogger<AllowHttpMethodsFilter> logger)
        {
            _logger = logger;
            // Constructor to inject the logger
            // Logger will be used to record information about blocked requests
        }
        public Task OnActionExecuting(ActionExecutingContext context)
        {
            // Log the incoming request
            _logger.LogInfo($"Incoming request: {context.HttpContext.Request.Method} {context.HttpContext.Request.Path}");
            // Log the HTTP method and path for monitoring
            var isAllowed = true;
            // Default to allowed
            var allowedMethods = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
            // Create a case-insensitive HashSet for allowed methods
            var controllerType = context.ActionDescriptor.ControllerTypeInfo;
            // Get the controller type information
            if (controllerType != null)
            {
                var methodInfo = controllerType.GetMethods();
                // Get all methods in the controller
                foreach (var method in methodInfo)
                {
                    var attributes = method.GetCustomAttributes();
                    // Get all attributes on the method
                    foreach (var attribute in attributes)
                    {
                        if (attribute is HttpMethodAttribute)
                        {
                            var httpMethod = ((HttpMethodAttribute)attribute).HttpMethods;
                            // Check if this is an HttpMethod attribute
                            if (httpMethod != null)
                            {
                                foreach (var m in httpMethod)
                                {
                                    allowedMethods.Add(m);
                                    // Add the allowed method to the list
                                }
                            }
                            else
                            {
                                // If no specific methods are specified, assume all methods are allowed
                                isAllowed = true;
                                break;
                            }
                        }
                        else
                        {
                            // If not an HttpMethod attribute, continue to next attribute
                            continue;
                        }
                    }
                }
                // Check if the current method is in the allowed methods
                var currentMethod = context.HttpContext.Request.Method;
                isAllowed = allowedMethods.Contains(currentMethod);
                // Check if the current method is allowed
                if (!isAllowed)
                {
                    _logger.LogWarning($"Blocked {currentMethod} request to {context.HttpContext.Request.Path}");
                    // Log the blocked request
                    var statusCode = 405;
                    var response = new { message = $"{currentMethod} Method not allowed" };
                    // Create response object
                    context.HttpContext.Response.StatusCode = statusCode;
                    context.HttpContext.Response.ContentType = "application/json";
                    // Set response headers
                    context.Result = new JsonResult(response);
                    // Return JSON response
                    return;
                    // Exit early if method is not allowed
                }
                else
                {
                    _logger.LogInfo($"Allowed {currentMethod} request to {context.HttpContext.Request.Path}");
                    // Log allowed requests for monitoring
                }
            }
            else
            {
                _logger.LogWarning("Controller type not found");
                // Log if controller type couldn't be found
                // This may happen with minimal APIs or misconfigured controllers
                isAllowed = true;
                // Default to allowed if controller type couldn't be found
                // This prevents accidentally blocking valid requests
            }
            return base.OnActionExecuting(context);
            // Call the base implementation to continue processing
            // This allows other filters to run if they exist
        }
        public Task OnActionExecuted(ActionExecutedContext context)
        {
            // Log the response status code
            var statusCode = context.HttpContext.Response.StatusCode;
            _logger.LogInfo($"Response status: {statusCode}");
            // Log the response status for monitoring
            return base.OnActionExecuted(context);
            // Call the base implementation
        }
    }
}
```

Note:

- **AllowHttpMethodsFilter** is an action filter specifically designed to restrict access based on the HTTP method used in the request.
- **OnActionExecuting** is implemented to intercept requests before they reach the controller action. It checks if the method is allowed, and if not, it sets the response to a 405 Method Not Allowed status code and prevents further processing.
- **OnActionExecuted** is called in this scenario so it is crucial to ensure all the blocked HTTP methods implementation is carefully specified in the controller or endpoints to prevent unintended exceptions.

Registering the Filter Globally:

Finally, add the following code in the Program.cs class file. By adding this filter, we are specifying which HTTP methods are allowed for our application.

```
builder.Services.AddControllers(options => options.Filters.Add<AllowHttpMethodsFilter>());
```

With the above changes in place, run the application and test each endpoint. It should work as expected in the previous example.

In the next article, I will discuss **How to Return Not Found Internal Server Error HTTP Status Code in ASP.NET Core Web API** with Examples. In this article, I hope you understand **How to Configure the Allowed HTTP Methods Globally in ASP.NET Core Web API** with Examples. I hope you enjoy this article.

Previous Lesson
Next Lesson
405 HTTP Status Code in ASP.NET Core Web API
500 HTTP Status Code in ASP.NET Core Web API

1 thought on "Configure Allowed HTTP Methods Globally in ASP.NET Core Web API"

Reply
Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment

ASP.NET Core Web API Tutorials
Introduction to ASP.NET Core Web API
Advantages and Disadvantages of ASP.NET Core Web API
Microsoft Dot Net Online Training
Microsoft Dot Net Online Training Tutorials
ASP.NET Core Web API Tutorials
Introduction to ASP.NET Core Web API
Advantages and Disadvantages of ASP.NET Core Web API
Microsoft Dot Net Online Training
Microsoft Dot Net Online Training Tutorials
Minimal and Lightweight Web Services in ASP.NET Core
Minimal and Lightweight Web Services in ASP.NET Core Tutorial
Minimal and Lightweight Web Services in ASP.NET Core Tutorial
Minimal and Lightweight Web Services in ASP.NET Core Tutorial